



VRSEC

VELAGAPUDI RAMAKRISHNA
SIDDHARTHA ENGINEERING COLLEGE
(Sponsored by Siddhartha Academy of General & Technical Education)

ONLINE QUIZ SYSTEM



B. TECH – COMPUTER SCIENCE AND
ENGINEERING

24CS202 – JAVA PROGRAMMING

HOME ASSIGNMENT
SUBMITTED BY,
24EU04307
24EU04324
24EU04325
SECOND YEAR

Online Quiz System

The **Online Quiz System** provides an interactive console-based platform where users can attempt quizzes, view the leaderboard, and check their performance. The system loads questions from a text file, displays them with multiple-choice options, tracks the user's score, and displays the final result. It also supports a timer for each question using a separate thread and saves quiz results and leaderboard data in files.

Implementations:

The system demonstrates the use of **Object-Oriented Programming (OOP)**, **Threads**, **File Handling**, and **Java Collection Framework** concepts effectively.

Package Structure:

- **quizsystem.main** - Contains the main entry point (Main) which controls menu navigation.
- **quizsystem.model** - Contains model classes for quiz entities like Question and LeaderboardEntry.
- **quizsystem.io** - Handles input/output operations such as loading questions and managing leaderboard files.
- **quizsystem.service** - Provides core quiz logic and the timer functionality using threads.
- **quizsystem.exceptions** - Defines custom exception classes for specific error handling.

List of Classes Used:

- **Main** - The main class that displays the menu and manages quiz flow.
- **Question** - Represents a single question with options, correct answer, and marks.

- **LeaderboardEntry** - Represents a user's name and score in the leaderboard.
- **QuestionLoader** - Loads quiz questions from the file.
- **LeaderboardManager** - Handles saving, loading, and displaying the leaderboard.
- **QuizService** - Core class that manages quiz execution, evaluation, and scoring.
- **TimerThread** - Inner class (extends Thread) that shows countdown time for each question.
- **QuestionFormatException** - Custom exception for malformed question files.
- **InvalidAnswerException** - Custom exception for invalid answer handling (future use).
- **ResultSaver** - Handles writing the user's performance summary (score, grade, and accuracy) to the result.txt file after quiz completion.

Files Used:

- **questions.txt** - Stores all quiz questions in a specific format
(id|text|option1|option2|option3|option4|correct|marks).
- **leaderboard.txt** - Stores usernames and their corresponding scores.
- **Result.txt** - Stores the detailed quiz result for each user after completion.

Features Implemented:

- Console-based quiz interaction.
- Dynamic scoring and grading system.
- Leaderboard display with sorted top scorers.

- Result file generation for user performance.
- Custom exception handling for invalid file or question formats

PROJECT STRUCTURE

OnlineQuizSystem

```
|
|
|─ quizsystem
|  |
|  |─ main
|  |  └─ Main.java
|  |
|  |─ model
|  |  └─ Question.java
|  |  └─ LeaderboardEntry.java
|  |
|  |─ service
|  |  └─ QuizService.java
|  |
|  |─ io
|  |  └─ QuestionLoader.java
|  |  └─ LeaderboardManager.java
|  |  └─ ResultSaver.java
|  |
|  └─ exceptions
|      └─ InvalidAnswerException.java
|      └─ QuestionFormatException.java
|
```

```
|— questions.txt  
|— leaderboard.txt  
└— result.txt
```

Question.java

```
package quizsystem.model;  
  
public class Question {  
    private int id;  
    private String text;  
    private String[] options;  
    private int correctOptionIndex;  
    private int marks;  
  
    public Question(int id, String text, String[] options, int  
correctOptionIndex, int marks) {  
        this.id = id;  
        this.text = text;  
        this.options = options;  
        this.correctOptionIndex = correctOptionIndex;  
        this.marks = marks;  
    }  
  
    public int getId() { return id; }  
    public String getText() { return text; }  
    public String[] getOptions() { return options; }  
    public int getCorrectOptionIndex() { return correctOptionIndex; }  
    public int getMarks() { return marks; }  
  
    public String getCorrectOption() {  
        int idx = Math.max(1, Math.min(4, correctOptionIndex));  
        char opt = (char) ('A' + idx - 1);
```

```
        return opt + ") " + options[idx - 1];  
    }  
}
```

LeaderboardEntry.java

```
package quizsystem.model;  
  
public class LeaderboardEntry {  
    private String username;  
    private int score;  
  
    public LeaderboardEntry(String username, int score) {  
        this.username = username;  
        this.score = score;  
    }  
  
    public String getUsername() { return username; }  
    public int getScore() { return score; }  
  
    @Override  
    public String toString() { return username + " - " + score; }  
}
```

QuestionFormatException.java

```
package quizsystem.exceptions;  
  
public class QuestionFormatException extends Exception {  
    public QuestionFormatException(String message) {  
        super(message);  
    }  
}
```

InvalidAnswerException.java

```
package quizsystem.exceptions;

public class InvalidAnswerException extends Exception {
    public InvalidAnswerException(String message) {
        super(message);
    }
}
```

QuestionLoader.java

```
package quizsystem.io;

import quizsystem.model.Question;
import quizsystem.exceptions.QuestionFormatException;
import java.io.*;
import java.util.*;

public class QuestionLoader {
    public static List<Question> loadQuestions(String fileName) throws
IOException, QuestionFormatException {
        List<Question> questions = new ArrayList<>();
        Set<Integer> ids = new HashSet<>();

        File file = new File(fileName);

        if (!file.exists()) throw new FileNotFoundException("Question file
not found: " + fileName);

        try (BufferedReader br = new BufferedReader(new FileReader(file))) {
            String line;
```

```

        int lineNo = 0;
        while ((line = br.readLine()) != null) {
            lineNo++;
            if (line.trim().isEmpty()) continue;
            String[] parts = line.split("\\|");
            if (parts.length != 8) {
                throw new QuestionFormatException("Malformed question at
line " + lineNo);
            }
            try {
                int id = Integer.parseInt(parts[0].trim());
                if (ids.contains(id)) continue;
                ids.add(id);

                String text = parts[1].trim();
                String[] options = new String[4];
                for (int i = 0; i < 4; i++) options[i] = parts[i +
2].trim();

                int correct = Integer.parseInt(parts[6].trim());
                int marks = Integer.parseInt(parts[7].trim());

                questions.add(new Question(id, text, options, correct,
marks));
            } catch (NumberFormatException ex) {
                throw new QuestionFormatException("Number format error
at line " + lineNo);
            }
        }
    }
    return questions;
}
}

```


LeaderboardManager.java

```
package quizsystem.io;

import quizsystem.model.LeaderboardEntry;
import java.io.*;
import java.util.*;

public class LeaderboardManager {

    public static Map<String, Integer> loadLeaderboardMap(String fileName)
    throws IOException {
        Map<String, Integer> map = new HashMap<>();
        File file = new File(fileName);
        if (!file.exists()) return map;

        try (BufferedReader br = new BufferedReader(new FileReader(file))) {
            String line;
            while ((line = br.readLine()) != null) {
                if (line.trim().isEmpty()) continue;
                String[] parts = line.split(":");
                if (parts.length == 2) {
                    try {
                        String user = parts[0].trim();
                        int score = Integer.parseInt(parts[1].trim());
                        map.put(user, score);
                    } catch (NumberFormatException ignored) { }
                }
            }
        }
    }
}
```

```

        return map;
    }

    public static void saveLeaderboardMap(String fileName, Map<String,
Integer> map) throws IOException {
        try (BufferedWriter bw = new BufferedWriter(new
FileWriter(fileName))) {
            for (Map.Entry<String, Integer> e : map.entrySet()) {
                bw.write(e.getKey() + ":" + e.getValue());
                bw.newLine();
            }
        }
    }

    public static void printSortedLeaderboard(Map<String, Integer> map) {
        List<LeaderboardEntry> entries = new ArrayList<>();
        for (Map.Entry<String, Integer> e : map.entrySet()) {
            entries.add(new LeaderboardEntry(e.getKey(), e.getValue()));
        }
        entries.sort((a, b) -> Integer.compare(b.getScore(), a.getScore()));
        if (entries.isEmpty()) {
            System.out.println("No entries in leaderboard yet.");
            return;
        }
        int rank = 1;
        for (LeaderboardEntry e : entries) {
            System.out.printf("%d. %s - %d%n", rank++, e.getUsername(),
e.getScore());
        }
    }
}

```

QuizService.java

```
package quizsystem.service;

import quizsystem.model.Question;
import quizsystem.io.ResultSaver;

import java.io.*;
import java.util.*;
import java.util.concurrent.atomic.AtomicBoolean;

public class QuizService {
    private List<Question> questions;
    private int score = 0, correct = 0, wrong = 0, skipped = 0;
    private List<Integer> userAnswers;
    private final int totalMarks;

    public QuizService(List<Question> questions) {
        this.questions = new ArrayList<>(questions); // preserve order
        this.userAnswers = new LinkedList<>();
        int sum = 0;
        for (Question q : questions) sum += q.getMarks();
        this.totalMarks = sum;
    }

    public void startQuiz(String username) {
        BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));

        System.out.println();

        for (Question q : questions) {
```

```

        System.out.println("-----
-");

        System.out.println("Question " + q.getId() + ": " +
q.getText());

        String[] opts = q.getOptions();
        System.out.println("A) " + opts[0]);
        System.out.println("B) " + opts[1]);
        System.out.println("C) " + opts[2]);
        System.out.println("D) " + opts[3]);
        System.out.print("Enter your answer (1-4) or any other key to
skip: ");

        String input = null;
        AtomicBoolean timedOut = new AtomicBoolean(false);

        try {
            for (int t = 60; t >= 0; t--) {
                // only show timer messages every 5 seconds to reduce
clutter

                if (t % 5 == 0 && t != 60) {
                    System.out.println("[TIMER] Time remaining: " + t +
" seconds");
                }
                if (br.ready()) {
                    input = br.readLine();
                    break;
                }
                if (t == 0) {
                    timedOut.set(true);
                    break;
                }
                Thread.sleep(1000);
            }
        } catch (Exception e) {

```

```

        // ignore interruptions
    }

    if (timedOut.get()) {
        System.out.println();
        System.out.println("[TIMEUP] Time is up for this
question!");
        System.out.println("Press ENTER to continue...");
        try {
            br.readLine(); // wait for user to press enter; treat as
skip
        } catch (IOException ignored) {}
        skipped++;
        userAnswers.add(0);
        System.out.println("-----"
-----");
        continue;
    }

    if (input == null || input.trim().isEmpty()) {
        skipped++;
        userAnswers.add(0);
        System.out.println("You skipped this question.");
    } else {
        input = input.trim();
        try {
            int ans = Integer.parseInt(input);
            if (ans >= 1 && ans <= 4) {
                char opt = (char) ('A' + ans - 1);
                System.out.println("You selected: " + opt + ") " +
opts[ans - 1]);
                userAnswers.add(ans);

                if (ans == q.getCorrectOptionIndex()) {

```

```

        correct++;

        score += q.getMarks();

    } else {

        wrong++;

    }

} else {

    skipped++;

    userAnswers.add(0);

    System.out.println("Invalid input. Marked as
skipped.");

}

} catch (NumberFormatException ex) {

    skipped++;

    userAnswers.add(0);

    System.out.println("Invalid input. Marked as skipped.");

}

}

// Pause briefly so the user sees the confirmation before next
question

try {

    Thread.sleep(1500);

} catch (InterruptedException e) {

    Thread.currentThread().interrupt();

}

} // end for

System.out.println();

System.out.println("===== QUIZ COMPLETED =====");

System.out.println(" Correct Answers : " + correct);

System.out.println(" Wrong Answers      : " + wrong);

System.out.println(" Skipped                : " + skipped);

System.out.println(" Total Score          : " + score);

```

```

        System.out.println("=====");

        showReviewSection();

        showPerformanceSummary();

        // Save the result to result.txt
        ResultSaver.saveResult(username, questions.size(), correct, wrong,
        skipped, score, totalMarks);
    }

    private void showReviewSection() {
        System.out.println();
        System.out.println("Review Section:");
        System.out.println("-----");
        for (int i = 0; i < questions.size(); i++) {
            Question q = questions.get(i);
            int userAns = 0;
            if (i < userAnswers.size()) userAns = userAnswers.get(i);

            System.out.println("Q" + q.getId() + ". " + q.getText());
            if (userAns == 0) {
                System.out.println("Your Answer    : Not Answered /
Skipped");
            } else {
                char opt = (char) ('A' + userAns - 1);
                System.out.println("Your Answer    : " + opt + ") " +
q.getOptions()[userAns - 1]);
            }
            System.out.println("Correct Answer: " + q.getCorrectOption());
            System.out.println("-----
-");
        }
    }
}

```

```

private void showPerformanceSummary() {
    double percentage = totalMarks == 0 ? 0.0 : ((double) score /
totalMarks) * 100.0;

    String rating;
    if (percentage >= 90) rating = "*****";
    else if (percentage >= 75) rating = "****";
    else if (percentage >= 50) rating = "***";
    else if (percentage >= 25) rating = "**";
    else rating = "*";

    System.out.println();
    System.out.println("-----");
    System.out.println("          QUIZ PERFORMANCE SUMMARY");
    System.out.println("-----");
    System.out.println("Your Rating : " + rating);
    System.out.printf("Your Accuracy: %.2f%%\n", percentage);
    if (percentage >= 75) {
        System.out.println("Great job! You have strong knowledge - keep
learning!");
    } else {
        System.out.println("Keep practicing to improve your accuracy.");
    }
    System.out.println("-----");
    System.out.println("Thank you for playing the Online Quiz System!");
    System.out.println("We hope you enjoyed testing your knowledge.");
    System.out.println("-----");
}

public int getScore() {
    return score;
}
}

```


Main.java

```
package quizsystem.main;

import quizsystem.io.QuestionLoader;
import quizsystem.io.LeaderboardManager;
import quizsystem.model.Question;
import quizsystem.service.QuizService;
import quizsystem.io.ResultSaver;
import quizsystem.exceptions.QuestionFormatException;

import java.io.*;
import java.util.*;

public class Main {

    private static final String QUESTION_FILE = "questions.txt";
    private static final String LEADERBOARD_FILE = "leaderboard.txt";

    private static String generateCandidateId() {
        long id = System.currentTimeMillis() % 1000000;
        return "U" + String.format("%06d", id);
    }

    public static void main(String[] args) {
        BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));

        while (true) {
            System.out.println("-----
-");

            System.out.println("                ONLINE QUIZ SYSTEM");

            System.out.println("-----
-");
```

```

        System.out.println("1. Start Quiz");
        System.out.println("2. View Leaderboard");
        System.out.println("3. Exit");
        System.out.println("-----
-");

        System.out.print("Enter your choice: ");
        try {
            String choice = br.readLine();
            if (choice == null) continue;
            switch (choice.trim()) {
                case "1":
                    startQuizFlow(br);
                    break;
                case "2":
                    viewLeaderboard();
                    break;
                case "3":
                    System.out.println("Thank you!");
                    return;
                default:
                    System.out.println("Invalid choice.");
            }
        } catch (IOException e) {
            System.out.println("Input error.");
        }
    }

    private static void startQuizFlow(BufferedReader br) {
        try {
            // Load existing leaderboard to check unique usernames

            Map<String, Integer> leaderboard =
LeaderboardManager.loadLeaderboardMap(LEADERBOARD_FILE);

```

```

Set<String> usernames = new HashSet<>(leaderboard.keySet());

System.out.print("\nEnter your name: ");
String username = br.readLine();
if (username == null) return;
username = username.trim();
while (username.isEmpty() || usernames.contains(username)) {
    if (username.isEmpty()) {
        System.out.print("Name cannot be empty. Enter your name:
");
    } else {
        System.out.print("Username already exists. Enter a
different name: ");
    }
    username = br.readLine();
    if (username == null) return;
    username = username.trim();
}

String candidateId = generateCandidateId();

System.out.println("Welcome, " + username + "!");
System.out.println();

// Welcome block exactly as requested (plain keyboard
characters)
System.out.println("-----
-");
System.out.println("          WELCOME TO THE ONLINE QUIZ SYSTEM
");
System.out.println("-----
-");
System.out.println("Sharpen your mind and challenge yourself!");
System.out.println("Answer smartly - every question tests your
knowledge and speed.");

```

```

        System.out.println("Each correct answer rewards you with marks;
think twice before skipping.");

        System.out.println("You have 60 seconds per question. Let's see
how high you can score!");

        System.out.println("-----
-");

        System.out.println();

        // countdown using Thread.sleep() (3..2..1..)
        try {
            System.out.println("Starting your quiz in 3...");
            Thread.sleep(1000);
            System.out.println("2...");
            Thread.sleep(1000);
            System.out.println("1...");
            Thread.sleep(1000);
            System.out.println("Good luck, " + username + "!");
            Thread.sleep(500);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }

        // show candidate id
        System.out.println("Your Candidate ID: " + candidateId);
        System.out.println();

        // Load questions in order
        List<Question> questions =
QuestionLoader.loadQuestions(QUESTION_FILE);
        if (questions.isEmpty()) {
            System.out.println("No questions found in " +
QUESTION_FILE);
            return;
        }

```

```

        // Start quiz
        QuizService quiz = new QuizService(questions);
        quiz.startQuiz(username);

        // Update leaderboard (keep best score)
        Map<String, Integer> map =
LeaderboardManager.loadLeaderboardMap(LEADERBOARD_FILE);

        int currentScore = quiz.getScore();
        if (map.containsKey(username)) {
            int existing = map.get(username);
            if (currentScore > existing) map.put(username,
currentScore);
        } else {
            map.put(username, currentScore);
        }
        LeaderboardManager.saveLeaderboardMap(LEADERBOARD_FILE, map);

        // Save result already handled inside QuizService via
ResultSaver

        // Display leaderboard
        System.out.println();
        System.out.println("Leaderboard:");
        LeaderboardManager.printSortedLeaderboard(map);
        System.out.println("-----
-");

        System.out.println("Leaderboard and results updated.");
        System.out.println("[PROGRAM END]");

    } catch (FileNotFoundException e) {
        System.out.println("Error: " + e.getMessage());
    } catch (QuestionFormatException e) {
        System.out.println("Error loading questions: " +
e.getMessage());
    }

```

```

    } catch (IOException e) {
        System.out.println("I/O error: " + e.getMessage());
    } catch (Exception e) {
        System.out.println("Unexpected error: " + e.getMessage());
    }
}

private static void viewLeaderboard() {
    try {
        Map<String, Integer> map =
LeaderboardManager.loadLeaderboardMap(LEADERBOARD_FILE);

        System.out.println();

        System.out.println("Leaderboard:");

        LeaderboardManager.printSortedLeaderboard(map);

        System.out.println("-----
-");
    } catch (IOException e) {
        System.out.println("Error loading leaderboard: " +
e.getMessage());
    }
}
}

```

Questions.txt

- 1|Which of the following is the superclass of all byte stream classes in Java?|Reader|Writer|InputStream|BufferedInputStream|3|2
- 2|Which class is used to read characters from a file in Java?|FileReader|FileInputStream|DataInputStream|ObjectInputStream|1|2
- 3|What is the main advantage of using BufferedReader over FileReader?|Reads binary data|Provides buffering for efficient reading|Automatically closes streams|Converts data into objects|2|2
- 4|Which method of InputStream returns -1 when the end of the file is reached?|available()|read()|skip()|reset()|2|2
- 5|Which of the following is used for object serialization in Java?|DataOutputStream|ObjectOutputStream|PrintWriter|BufferedOutputStream|2|2
- 6|Which of the following classes is used to write primitive data types into an output stream?|ObjectOutputStream|BufferedWriter|DataOutputStream|FileWriter|3|2
- 7|Which method of Reader is used to close the stream?|end()|stop()|close()|exit()|3|1
- 8|Which class provides methods for reading formatted input in Java?|PrintWriter|Scanner|BufferedReader|DataInputStream|2|1
- 9|What does FileWriter in Java primarily handle?|Reading characters|Writing characters|Reading bytes|Writing bytes|2|1
- 10|Which exception is most commonly associated with Input/Output operations in Java?|NullPointerException|IOException|ClassNotFoundException|ArithmeticException|2|1

OUTPUT :

Question 1: Which of the following is the superclass of all byte stream classes in Java?

- A) Reader
- B) Writer
- C) InputStream
- D) BufferedInputStream

Enter your answer (1-4) or any other key to skip: 1

You selected: A) Reader

Question 2: Which class is used to read characters from a file in Java?

- A) FileReader
- B) FileInputStream
- C) DataInputStream
- D) ObjectInputStream

Enter your answer (1-4) or any other key to skip: 2

You selected: B) FileInputStream

Question 3: What is the main advantage of using BufferedReader over FileReader?

- A) Reads binary data
- B) Provides buffering for efficient reading
- C) Automatically closes streams
- D) Converts data into objects

Enter your answer (1-4) or any other key to skip: 3

You selected: C) Automatically closes streams

Question 4: Which method of InputStream returns -1 when the end of the file is reached?

- A) available()
- B) read()
- C) skip()
- D) reset()

Enter your answer (1-4) or any other key to skip: 4

You selected: D) reset()

Question 5: Which of the following is used for object serialization in Java?

- A) DataOutputStream
- B) ObjectOutputStream
- C) PrintWriter
- D) BufferedOutputStream

Enter your answer (1-4) or any other key to skip: 1

You selected: A) DataOutputStream

Question 6: Which of the following classes is used to write primitive data types into an output stream?

- A) ObjectOutputStream
- B) BufferedWriter
- C) DataOutputStream
- D) FileWriter

Enter your answer (1-4) or any other key to skip: 2

You selected: B) BufferedWriter

Question 7: Which method of Reader is used to close the stream?

- A) end()
- B) stop()
- C) close()
- D) exit()

Enter your answer (1-4) or any other key to skip: 3

You selected: C) close()

Question 8: Which class provides methods for reading formatted input in Java?

- A) PrintWriter
- B) Scanner
- C) BufferedReader
- D) DataInputStream

Enter your answer (1-4) or any other key to skip: 0

Invalid input. Marked as skipped.

Question 9: What does FileWriter in Java primarily handle?

- A) Reading characters
- B) Writing characters
- C) Reading bytes
- D) Writing bytes

Enter your answer (1-4) or any other key to skip: [TIMER] Time remaining: 55 seconds

[TIMER] Time remaining: 50 seconds

[TIMER] Time remaining: 45 seconds

[TIMER] Time remaining: 40 seconds

[TIMER] Time remaining: 35 seconds

[TIMER] Time remaining: 30 seconds

[TIMER] Time remaining: 25 seconds

[TIMER] Time remaining: 20 seconds

[TIMER] Time remaining: 15 seconds

[TIMER] Time remaining: 10 seconds

[TIMER] Time remaining: 5 seconds

[TIMER] Time remaining: 0 seconds

[TIMEUP] Time is up for this question!

Press ENTER to continue...

1

Question 10: Which exception is most commonly associated with Input/Output operations in Java?

- A) NullPointerException
- B) IOException
- C) ClassNotFoundException
- D) ArithmeticException

Enter your answer (1-4) or any other key to skip: [TIMER] Time remaining: 55 seconds

[TIMER] Time remaining: 50 seconds

15

Invalid input. Marked as skipped.

===== QUIZ COMPLETED =====

Correct Answers : 1

Wrong Answers : 6

Skipped : 3

Total Score : 1

Review Section:

Q1. Which of the following is the superclass of all byte stream classes in Java?

Your Answer : A) Reader

Correct Answer: C) InputStream

Q2. Which class is used to read characters from a file in Java?

Your Answer : B) FileInputStream

Correct Answer: A) FileReader

Q3. What is the main advantage of using BufferedReader over FileReader?

Your Answer : C) Automatically closes streams

Correct Answer: B) Provides buffering for efficient reading

Q4. Which method of InputStream returns -1 when the end of the file is reached?

Your Answer : D) reset()

Correct Answer: B) read()

Q5. Which of the following is used for object serialization in Java?

Your Answer : A) DataOutputStream

Correct Answer: B) ObjectOutputStream

Q6. Which of the following classes is used to write primitive data types into an output stream?

Your Answer : B) BufferedWriter

Correct Answer: C) DataOutputStream

Q7. Which method of Reader is used to close the stream?

Your Answer : C) close()

Correct Answer: C) close()

Q8. Which class provides methods for reading formatted input in Java?

Your Answer : Not Answered / Skipped

Correct Answer: B) Scanner

Q9. What does FileWriter in Java primarily handle?

Your Answer : Not Answered / Skipped

Correct Answer: B) Writing characters

Q10. Which exception is most commonly associated with Input/Output operations in Java?

Your Answer : Not Answered / Skipped

Correct Answer: B) IOException

QUIZ PERFORMANCE SUMMARY

Your Rating : *
Your Accuracy: 6.25%
Keep practicing to improve your accuracy.

Thank you for playing the Online Quiz System!
We hope you enjoyed testing your knowledge.

Leaderboard:
1. Kameswar - 2
2. Hari - 1
3. Balu - 1

Leaderboard and results updated.
[PROGRAM END]

ONLINE QUIZ SYSTEM

1. Start Quiz
2. View Leaderboard
3. Exit

Enter your choice: 2

Leaderboard:
1. Kameswar - 2
2. Hari - 1
3. Balu - 1

ONLINE QUIZ SYSTEM

1. Start Quiz
2. View Leaderboard
3. Exit

Enter your choice: |

Leaderboard.txt

Hari:1

Balu:1

Kameswar:2

Result.txt

User: Hari

Total Questions: 10

Correct Answers: 1

Wrong Answers: 6

Skipped: 3

Score: 1 / 16

Percentage: 6.25%

Grade: D

Driver's Link : <https://drive.google.com/drive/folders/1JJnqu6xFxI0OX0-difdD0uHSybahGj3b?usp=sharing>